Replit Utvecklingsprompt: Fält App (Driver Core) - AI-Driven Mobil App för Fälttekniker

---

1. Executive Summary

Vad är Fält App?

Fält App (Driver Core) är en **native mobilapplikation** byggd med React Native/Expo för fälttekniker och chaufförer inom fältservice (avfallshantering, städning, fastighetsservice). Appen är den primära arbetsmiljön för tekniker i fält och ska transformeras från ett transaktionellt verktyg till en **intelligent AI-driven arbetsassistent**.

Målgrupp

- **Primär:** Fälttekniker och chaufförer (80% av användartiden)
- **Sekundär:** Teamledare i fält
- **Profil:** Praktiskt lagda, ofta äldre, begränsad teknisk erfarenhet, arbetar med handskar, utomhus i alla väder

Huvudsyfte

Effektivisera fältarbete genom:
1. **Hands-free operationer** - Röststyrning för säkerhet och effektivitet
2. **Automatisk dokumentation** - AI analyserar bilder och skapar rapporter
3. **Proaktiv assistans** - AI föreslår nästa steg baserat på kontext
4. **Minimal input** - One-touch-rapportering med AI som fyller i detaljerna

Affärsmål

Mätetal	Nuvarande	Mål (12 mån)	Förbättring
Tid för dokumentation	3-5 min/uppgift	30 sek/uppgift	**85%**
Hands-free tid	0%	90%	∞
Felrapportering	5-10% missar	<1%	**90%**
Teknikerutbildning	6 månader	2 veckor	**85%**

---

2. Nuvarande Funktionalitet (Behåll/Förbättra)

Stack och Arkitektur

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        DRIVER CORE (Fält App)                            │
├──────────────────────────────────────────────────────────────────────────┤
│  React Native + Expo SDK 54     │    TypeScript                          │
│  React Navigation 7             │    @tanstack/react-query               │
│  Inter Font                     │    Native Maps + Fallback              │
├──────────────────────────────────────────────────────────────────────────┤
│                            API Layer                                     │
│  POST /api/mobile/login        GET /api/mobile/orders                    │
│  PATCH /orders/:id/status      POST /orders/:id/inspections              │
│  POST /orders/:id/notes        POST /api/mobile/gps                      │
├──────────────────────────────────────────────────────────────────────────┤
│                       Backend (Express.js:5000)                          │
│                       PostgreSQL Database                                │
└──────────────────────────────────────────────────────────────────────────┘
```

Befintliga Funktioner (✅ Behåll)

Funktion	Status	Beskrivning
PIN-login	✅ Fullständig	4-6 siffror + användarnamn/lösenord fallback
Orderlista	✅ Fullständig	Filtrering, färgkodade status-badges
Status-workflow	✅ Fullständig	8-stegs workflow med haptisk feedback
Task dependencies	✅ Fullständig	Visuella lås, röd-tonade kort för låsta
Execution codes	✅ Fullständig	Färgkodade badges (TÖM, HÄMT, FARL)
Time restrictions	✅ Fullständig	Varningar för tidsfönster
Sub-steps	✅ Fullständig	Expanderbar checklista med progress
Inspektioner	✅ Fullständig	6 kategorier, OK/Varning/Fel
Order notes	✅ Fullständig	Textbaserade anteckningar
GPS-spårning	✅ Fullständig	Realtidsposition till backend
Offline-indikator	⚠️ Grundläggande	Nätverksstatus visas
Kamera	⚠️ Manuell	Bildtagning utan AI-analys
Navigation	⚠️ Grundläggande	Öppnar extern app
Väderinfo	✅ Fullständig	Open-Meteo med varningar
Avvikelserapport	✅ Fullständig	GPS + manuell beskrivning
Materialogg	✅ Fullständig	Artikel-autocomplete
Signatur	✅ Fullständig	Digital signatur
What3Words	✅ Fullständig	Positionsvisning

Directory-struktur

```
/
├── App.tsx
├── app.json
├── scripts/build.sh
├── dist/
├── client/
│   ├── components/
│   │   ├── Card.tsx
│   │   ├── ErrorBoundary.tsx
│   │   ├── OfflineIndicator.tsx
│   │   ├── StatusBadge.tsx
│   │   ├── ThemedText.tsx
│   │   └── ThemedView.tsx
│   ├── constants/theme.ts
│   ├── context/AuthContext.tsx
│   ├── hooks/useScreenOptions.ts
│   ├── lib/query-client.ts
│   ├── navigation/
│   │   ├── RootNavigator.tsx
│   │   └── TabNavigator.tsx
│   ├── screens/
│   │   ├── LoginScreen.tsx
│   │   ├── HomeScreen.tsx
│   │   ├── OrdersScreen.tsx
│   │   ├── OrderDetailScreen.tsx
│   │   ├── InspectionScreen.tsx
│   │   ├── MapScreen.tsx
│   │   ├── ProfileScreen.tsx
│   │   ├── ReportDeviationScreen.tsx
│   │   ├── MaterialLogScreen.tsx
│   │   ├── CameraCaptureScreen.tsx
│   │   └── SignatureScreen.tsx
│   └── types/index.ts
├── server/
│   ├── index.ts
│   ├── routes/mobile.ts
│   └── templates/landing-page.html
└── assets/
```

---

3. AI-Drivna Funktioner - PRIORITERADE

KRITISK PRIORITET (Fas 1 - Månad 1-3)

Funktion 1: AI-Assistent för Fälttekniker (Röststyrd)

Detaljerad Beskrivning

En hands-free röstassistent som tekniker kan prata med för att:
- Ställa frågor om uppgifter (“Hur många kärl?”, “Vad är koden?”)
- Rapportera avvikelser (“Jag ser trasigt lysrör”)
- Skapa uppgifter (“Boka byte nästa vecka”)

- Navigera i appen (“Visa nästa uppgift”)
- Få kontext om objekt (“Vad gjordes senast här?”)

Teknisk Implementation

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        RÖSTSTYRD AI-ASSISTENT ARKITEKTUR                 │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐       │
│   │ Mikrofon │──►│ Whisper  │──►│  GPT-4   │──►│ ElevenLabs│       │
│   │ (Native) │     │  (STT)   │     │ (Reason) │     │   (TTS)  │       │
│   └──────────┘     └──────────┘     └──────────┘     └──────────┘       │
│        ▲                                  │                 │           │
│        │                                  ▼                 │           │
│        │                     ┌──────────────────────┐       │           │
│        │                     │    UNICORN CONTEXT   │       │           │
│        │                     ├──────────────────────┤       ▼           │
│        │                     │ • Ordrar & uppgifter │  ┌──────────┐     │
│        └─────────────────────│ • Metadata (portkoder) │  │ Högtalare │     │
│          Wake word           │ • Objekthistorik     │  │ (Native) │     │
│          "Hej Unicorn"       │ • Arbetsinstruktioner │  └──────────┘     │
│                              │ • Kundinformation    │                   │
│                              └──────────────────────┘                   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

API-integration

OpenAI Whisper (Speech-to-Text)

```
// client/services/voice-assistant.ts

import { Audio } from 'expo-av';

export async function transcribeAudio(audioUri: string): Promise<string> {
  const formData = new FormData();
  formData.append('file', {
    uri: audioUri,
    type: 'audio/m4a',
    name: 'recording.m4a',
  } as any);
  formData.append('model', 'whisper-1');
  formData.append('language', 'sv');

  const response = await fetch('/api/mobile/voice/transcribe', {
    method: 'POST',
    body: formData,
    headers: { 'Authorization': `Bearer ${authToken}` },
  });

  return (await response.json()).text;
}
```

GPT-4 (Reasoning)

```
// server/services/ai-assistant.ts

export async function processVoiceQuery(
  transcript: string,
  context: TechnicianContext
): Promise<AssistantResponse> {
  const systemPrompt = `
    Du är en AI-assistent för fälttekniker på ${context.companyName}.

    KONTEXT:
    - Tekniker: ${context.technicianName}
    - Nuvarande uppgift: ${context.currentOrder?.description || 'Ingen'}
    - Objekt: ${context.objectInfo?.address || 'Okänt'}
    - Portkod: ${context.objectInfo?.metadata?.portkod || 'Ej registrerad'}
    - Historik: ${JSON.stringify(context.recentHistory)}

    REGLER:
    1. Svara koncist på svenska
    2. Om du inte vet svaret, säg det
    3. Föreslå alltid nästa steg
    4. Om tekniker rapporterar fel, fråga om de vill skapa uppgift

    VERKTYG DU KAN ANVÄNDA:
    - create_deviation: Skapa avvikelserapport
    - create_task: Skapa ny uppgift
    - update_metadata: Uppdatera objektinfo
    - navigate_to: Navigera i appen
  `;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [
      { role: 'system', content: systemPrompt },
      { role: 'user', content: transcript }
    ],
    tools: UNICORN_TOOLS,
    temperature: 0.7,
  });

  return parseAssistantResponse(response);
}
```

ElevenLabs (Text-to-Speech)

```
// server/services/tts.ts

export async function generateSpeech(text: string): Promise<Buffer> {
  const response = await fetch('https://api.elevenlabs.io/v1/text-to-speech/svenska-
man', {
    method: 'POST',
    headers: {
      'xi-api-key': process.env.ELEVENLABS_API_KEY!,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      text,
      model_id: 'eleven_multilingual_v2',
      voice_settings: {
        stability: 0.5,
        similarity_boost: 0.75,
      }
    }),
  });

  return Buffer.from(await response.arrayBuffer());
}
```

Nya API-Endpoints

```
// server/routes/mobile.ts - Lägg till:

// POST /api/mobile/voice/transcribe - Transkribera ljud
// POST /api/mobile/voice/query - Skicka fråga till AI
// GET /api/mobile/voice/stream - SSE för AI-svar
// POST /api/mobile/voice/action - Utför AI-föreslagen åtgärd
```

React Native Komponenter

```
// client/components/VoiceAssistant.tsx

import React, { useState, useEffect } from 'react';
import { View, TouchableOpacity, Text, StyleSheet } from 'react-native';
import { Audio } from 'expo-av';
import { Ionicons } from '@expo/vector-icons';
import * as Haptics from 'expo-haptics';

interface VoiceAssistantProps {
  currentOrderId?: string;
  onAction?: (action: AIAction) => void;
}

export function VoiceAssistant({ currentOrderId, onAction }: VoiceAssistantProps) {
  const [isListening, setIsListening] = useState(false);
  const [transcript, setTranscript] = useState('');
  const [response, setResponse] = useState('');
  const [isProcessing, setIsProcessing] = useState(false);

  const startListening = async () => {
    await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium);
    setIsListening(true);
    // Start recording...
  };

  const stopListening = async () => {
    setIsListening(false);
    setIsProcessing(true);
    // Stop recording, transcribe, send to AI...
  };

  return (
    <View style={styles.container}>
      {/* Floating mic button */}
      <TouchableOpacity
        style={[styles.micButton, isListening && styles.micButtonActive]}
        onPressIn={startListening}
        onPressOut={stopListening}
      >
        <Ionicons
          name={isListening ? "mic" : "mic-outline"}
          size={32}
          color="white"
        />
      </TouchableOpacity>

      {/* Transcript bubble */}
      {transcript && (
        <View style={styles.transcriptBubble}>
          <Text style={styles.transcriptText}>{transcript}</Text>
        </View>
      )}

      {/* AI Response bubble */}
      {response && (
        <View style={styles.responseBubble}>
          <Text style={styles.responseText}>{response}</Text>
        </View>
      )}

      {/* Processing indicator */}
      {isProcessing && (
```

```
        <View style={styles.processingIndicator}>
          <ActivityIndicator color="#1B4F72" />
          <Text>Tänker...</Text>
        </View>
      )}
    </View>
  );
}
```

Användarflöden

Flöde 1.1: Fråga om information

```
1. Tekniker håller inne mikrofon-knappen
2. Säger: "Vad är portkoden?"
3. Släpper knappen
4. [Whisper transkriberar: "Vad är portkoden?"]
5. [GPT-4 hämtar metadata för aktuellt objekt]
6. [ElevenLabs genererar tal]
7. AI svarar via högtalare: "Portkoden är 5678. Den uppdaterades förra veckan."
8. Svar visas också som text på skärmen
```

Flöde 1.2: Rapportera fel

```
1. Tekniker: "Jag ser att gasdämparen är trasig"
2. AI: "Jag noterar det. Vill du:
        1. Ta en bild?
        2. Skapa en uppgift för byte?
        3. Båda?"
3. Tekniker: "Båda"
4. [Kamera öppnas automatiskt]
5. Tekniker tar bild
6. AI: "Bild sparad. Jag har skapat uppgift 'Byt gasdämpare' på Palmgatan 17.
        Vill du lägga till den på din rutt?"
7. Tekniker: "Ja, nästa vecka"
8. AI: "Klart! Uppgift schemalagd för nästa vecka."
```

Flöde 1.3: Navigation i appen

```
1. Tekniker: "Visa nästa uppgift"
2. AI: "Nästa uppgift är på Storgatan 5, 2 km bort.
        Tvätta 3 kärl. Vill du att jag navigerar dit?"
3. Tekniker: "Ja"
4. [Google Maps/Apple Maps öppnas med rutt]
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Rösttranskription	95% korrekt	Manuell validering av 100 fraser
Svarstid (total)	< 3 sekunder	Automatisk mätning
Bullerhantering	Fungerar vid 70+ dB	Test i verklig miljö
Hands-free	Ingen touch krävs efter aktiv-ering	Observationstest
Kontextförståelse	90% korrekta svar	Manuell validering
Åtgärdsutförande	100% korrekta åtgärder	Automatisk validering
Svenskt tal	Naturligt uttal	Användarfeedback

Funktion 2: AI-Driven Bildanalys och Automatisk Rapportering

Detaljerad Beskrivning

Tekniker tar bilder → AI analyserar automatiskt:
- Identifierar objekt (antal kärl, typ, färg)
- Upptäcker avvikelser (trasiga delar, skador, fel)
- Läser text (serienummer, etiketter)
- Jämför före/efter
- Genererar strukturerad rapport

Teknisk Implementation

```
BILDANALYS-PIPELINE

Kamera (Native) → Compress + Upload → Vision API → Report Generator
                                       ↓
                                  ANALYS-OUTPUT
                                  • objects_detected: [
                                      { type: "kärl", count: 4 },
                                      { type: "lock", status: ok }
                                    ]
                                  • anomalies: [
                                      { type: "trasigt_lysrör" }
                                    ]
                                  • text_detected: "240L"
                                  • before_after_diff: [...]
```

API-integration (OpenAI Vision)

```
// server/services/vision-analysis.ts

interface VisionAnalysisResult {
  objectsDetected: DetectedObject[];
  anomalies: Anomaly[];
  textDetected: string[];
  condition: 'excellent' | 'good' | 'fair' | 'poor';
  confidence: number;
  suggestedMetadataUpdates: MetadataUpdate[];
}

export async function analyzeImage(
  imageBase64: string,
  context: AnalysisContext
): Promise<VisionAnalysisResult> {
  const systemPrompt = `
    Du är en expert på att analysera bilder från fältservice.

    KONTEXT:
    - Objekttyp: ${context.objectType} (t.ex. återvinningsrum, molok)
    - Förväntat antal kärl: ${context.expectedContainerCount}
    - Senaste besiktning: ${context.lastInspection}

    ANALYSERA:
    1. Räkna exakt antal kärl per typ (matavfall, plast, kartong, etc.)
    2. Identifiera skador eller avvikelser:
       - Trasiga lock, gasdämpare, hjul
       - Fläckar, skador, graffiti
       - Felplacerade kärl
    3. Läs synlig text (serienummer, volymetiketter)
    4. Bedöm övergripande skick (1-10)

    RETURNERA JSON med exakt detta format:
    {
      "objectsDetected": [{ "type": "string", "count": number, "details": "string" }],
      "anomalies": [{ "type": "string", "severity": "low|medium|high", "description":
"string" }],
      "textDetected": ["string"],
      "condition": "excellent|good|fair|poor",
      "confidence": number (0-1),
      "suggestedMetadataUpdates": [{ "field": "string", "value": "any" }]
    }
  `;

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [
      { role: 'system', content: systemPrompt },
      {
        role: 'user',
        content: [
          { type: 'text', text: 'Analysera denna bild från fältbesök:' },
          { type: 'image_url', image_url: { url: `$
{imageBase64}` } }
        ]
      }
    ],
    response_format: { type: 'json_object' },
    max_tokens: 1000,
  });
```

```
  return JSON.parse(response.choices[0].message.content!);
}
```

Före/Efter-jämförelse

```
// server/services/before-after-comparison.ts

export async function compareBeforeAfter(
  beforeImage: string,
  afterImage: string,
  context: ComparisonContext
): Promise<ComparisonResult> {
  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [
      {
        role: 'system',
        content: `
          Jämför dessa två bilder (före och efter service).
          Identifiera:
          1. Vad som är ändrat/förbättrat
          2. Kvarstående problem
          3. Kvalitetspoäng (1-10) för utfört arbete
        `
      },
      {
        role: 'user',
        content: [
          { type: 'text', text: 'FÖRE:' },
          { type: 'image_url', image_url: { url: `$
{beforeImage}` } },
          { type: 'text', text: 'EFTER:' },
          { type: 'image_url', image_url: { url: `${afterImage}
` } }
        ]
      }
    ],
    response_format: { type: 'json_object' },
  });

  return JSON.parse(response.choices[0].message.content!);
}
```

Automatisk Metadata-uppdatering

```
// server/services/auto-metadata.ts

export async function updateMetadataFromAnalysis(
  objectId: string,
  analysis: VisionAnalysisResult
): Promise<MetadataUpdateResult> {
  const updates: MetadataUpdate[] = [];

  // Uppdatera antal kärl om det skiljer sig
  if (analysis.objectsDetected.length > 0) {
    const containerCount = analysis.objectsDetected
      .filter(o => o.type.includes('kärl'))
      .reduce((sum, o) => sum + o.count, 0);

    updates.push({
      field: 'antal_karl',
      value: containerCount,
      confidence: analysis.confidence,
      source: 'vision_ai'
    });
  }

  // Uppdatera skick
  updates.push({
    field: 'senaste_skickbedomning',
    value: analysis.condition,
    confidence: analysis.confidence,
    source: 'vision_ai'
  });

  // Spara med audit trail
  return await metadataService.batchUpdate(objectId, updates, {
    method: 'ai_vision_analysis',
    timestamp: new Date(),
  });
}
```

React Native Komponenter

```
// client/screens/SmartCaptureScreen.tsx

export function SmartCaptureScreen({ route, navigation }) {
  const { orderId, captureType } = route.params; // 'before' | 'after' | 'deviation'
  const [analysis, setAnalysis] = useState<VisionAnalysisResult | null>(null);
  const [isAnalyzing, setIsAnalyzing] = useState(false);

  const handleCapture = async (photo: CameraCapturedPicture) => {
    setIsAnalyzing(true);

    try {
      // 1. Komprimera bild
      const compressed = await compressImage(photo.uri, { quality: 0.8 });

      // 2. Skicka för analys
      const result = await api.post('/api/mobile/vision/analyze', {
        image: compressed.base64,
        orderId,
        captureType,
      });

      setAnalysis(result.data);

      // 3. Visa resultat för bekräftelse
      await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);

    } catch (error) {
      Alert.alert('Analysfel', 'Kunde inte analysera bilden. Försök igen.');
    } finally {
      setIsAnalyzing(false);
    }
  };

  return (
    <View style={styles.container}>
      <Camera onCapture={handleCapture} />

      {isAnalyzing && (
        <View style={styles.analyzingOverlay}>
          <LottieView source={require('../assets/analyzing.json')} autoPlay />
          <Text>AI analyserar bilden...</Text>
        </View>
      )}

      {analysis && (
        <AnalysisResultSheet
          result={analysis}
          onConfirm={() => handleConfirm(analysis)}
          onRetake={() => setAnalysis(null)}
          onEdit={() => navigation.navigate('EditAnalysis', { analysis })}
        />
      )}
    </View>
  );
}
```

Användarflöden

Flöde 2.1: Standard bildanalys

```
1. Tekniker trycker "Ta bild" på OrderDetailScreen
2. Kamera öppnas med AI-indikator
3. Tekniker tar bild
4. [AI analyserar i 3-5 sekunder]
5. Resultat visas:
   ┌──────────────────────────────┐
   │ 🤖 AI-ANALYS                 │
   ├──────────────────────────────┤
   │ Identifierat:                │
   │ • 4 st matavfallskärl (240L) ✓│
   │ • 2 st plastkärl (240L) ✓    │
   │                              │
   │ Avvikelser:                  │
   │ ⚠️ Trasigt lock på kärl #3   │
   │                              │
   │ Skick: Bra (7/10)            │
   │                              │
   │ [Godkänn] [Justera] [Ta ny bild]│
   └──────────────────────────────┘
6. Tekniker trycker "Godkänn"
7. Bild sparas med AI-genererad metadata
```

Flöde 2.2: Före/efter-jämförelse

```
1. Tekniker har tagit "före"-bild vid start
2. Efter utfört arbete, trycker "Ta efterbild"
3. [AI jämför före/efter]
4. Resultat:
   ┌──────────────────────────────┐
   │ 🤖 FÖRE/EFTER-JÄMFÖRELSE     │
   ├──────────────────────────────┤
   │ [FÖRE-bild]      [EFTER-bild]│
   │                              │
   │ Förändringar:                │
   │ ✅ Kärl tvättade (synligt renare)│
   │ ✅ Lock öppnas nu (var fastfruset)│
   │ ⚠️ Trasigt lock kvarstår     │
   │                              │
   │ Kvalitet: 9/10               │
   │                              │
   │ [Skicka rapport]             │
   └──────────────────────────────┘
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Objektigenkänning	85% noggrannhet	Jämför med manuell räkning
Feldetektering	80% upptäckt	Jämför med expertkontroll
Analysttid	< 5 sekunder	Automatisk mätning
Metadata-uppdatering	100% korrekt format	Automatisk validering
Före/efter-jämförelse	90% korrekta slutsatser	Manuell validering

Funktion 3: Automatisk Kundkommunikation (Tekniker-sida)

Detaljerad Beskrivning

Tekniker behöver inte manuellt informera kunder. Systemet skickar automatiskt:
- SMS/Email vid start av resa
- ETA-uppdateringar
- Meddelande vid ankomst
- Rapport med bilder vid slutförande
- Uppföljning vid upptäckta fel

Teknisk Implementation

```
// server/services/customer-communication.ts

export class CustomerCommunicationService {
  async handleOrderStatusChange(
    orderId: string,
    newStatus: OrderStatus,
    context: StatusChangeContext
  ) {
    const order = await getOrder(orderId);
    const customer = await getCustomer(order.customerId);
    const preferences = await getNotificationPreferences(customer.id);

    if (!preferences.enableSms) return;

    const message = await this.generateMessage(order, newStatus, context);

    await this.sendSms(customer.phone, message);
    await this.logCommunication(orderId, 'sms', message);
  }

  private async generateMessage(
    order: Order,
    status: OrderStatus,
    context: StatusChangeContext
  ): Promise<string> {
    const templates: Record<OrderStatus, string> = {
      'on_route': `${context.technicianName} är på väg till er. Beräknad ankomst: ${co
ntext.eta}. Spåra: ${context.trackingUrl}`,
      'arrived': `${context.technicianName} har anlänt!`,
      'completed': `Klart! ${order.description}. Se bilder: ${context.reportUrl}`,
      'deviation_found': `Vi upptäckte: ${context.deviationDescription}. Vill du att
vi åtgärdar? (${context.estimatedCost}kr) Svara JA/NEJ`,
    };

    return templates[status] || '';
  }
}
```

Event-triggers i Fält App

```
// client/hooks/useOrderStatusWithNotifications.ts

export function useOrderStatusWithNotifications(orderId: string) {
  const mutation = useMutation({
    mutationFn: async (newStatus: OrderStatus) => {
      // 1. Uppdatera status
      const result = await api.patch(`/api/mobile/orders/${orderId}/status`, {
        status: newStatus,
        timestamp: new Date().toISOString(),
        gpsPosition: await getCurrentPosition(),
      });

      // 2. Backend triggar automatiskt kundkommunikation
      // (Ingen extra kod behövs i appen)

      return result;
    },
  });

  return mutation;
}
```

Användarflöden

Flöde 3.1: Automatisk statusuppdatering

```
1. Tekniker trycker "Starta resa" för order
2. App uppdaterar status till "on_route"
3. Backend beräknar ETA baserat på GPS + trafik
4. Kund får automatiskt SMS:
   "Johan är på väg. Ankomst ca 10:15. Följ: [länk]"
5. Vid ankomst (GPS inom 50m):
   Kund får SMS: "Johan har anlänt!"
```

Flöde 3.2: Rapport med bilder

```
1. Tekniker slutför uppgift med bilder
2. Backend genererar rapport automatiskt
3. Kund får SMS:
   "Tvätt klar! 4 kärl tvättade. Se bilder och rapport: [länk]"
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
SMS skickas inom	< 30 sekunder	Automatisk mätning
Korrekt ETA	±5 minuter	Jämför med faktisk ankomst
Länk fungerar	100%	Automatisk kontroll
Kund kan svara	JA/NEJ hanteras	Test

Funktion 4: “One-Touch Reporting”

Detaljerad Beskrivning

Minimal input från tekniker = maximal output. En knapptryckning genererar komplett rapport med:

- AI-analyserade bilder
- Automatisk metadata-uppdatering
- Genererad sammanfattning
- Kundsignatur (om krävs)
- Faktureringsunderlag

Teknisk Implementation

```
// server/services/one-touch-report.ts

export async function generateOneTouchReport(
  orderId: string,
  images: ImageAnalysis[],
  context: ReportContext
): Promise<OneTouchReport> {
  // 1. Samla all data
  const order = await getOrder(orderId);
  const object = await getObject(order.objectId);
  const inspections = await getInspections(orderId);
  const materials = await getMaterialsUsed(orderId);
  const timeSpent = calculateTimeSpent(order);

  // 2. AI genererar sammanfattning
  const summary = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [
      {
        role: 'system',
        content: 'Generera en professionell sammanfattning av utfört arbete. Max 3
meningar.'
      },
      {
        role: 'user',
        content: JSON.stringify({
          description: order.description,
          objectType: object.type,
          imagesAnalysis: images.map(i => i.analysis),
          inspections,
          materials,
          timeSpent,
        })
      }
    ]
  });

  // 3. Skapa rapport
  return {
    orderId,
    generatedAt: new Date(),
    summary: summary.choices[0].message.content,
    details: {
      workDescription: order.description,
      objectAddress: object.address,
      timeSpent,
      materialsUsed: materials,
      inspectionResults: inspections,
    },
    images: images.map(i => ({
      url: i.url,
      type: i.type,
      aiAnalysis: i.analysis,
    })),
    metadata: {
      technicianName: context.technicianName,
      completedAt: new Date(),
      gpsPosition: context.gpsPosition,
      weatherConditions: await getWeather(context.gpsPosition),
    },
    invoiceData: calculateInvoiceData(order, materials, timeSpent),
```

```
  };
}
```

React Native Implementation

```
// client/screens/OrderDetailScreen.tsx - Ny "Klar"-knapp

function CompletionButton({ orderId }) {
  const [isGenerating, setIsGenerating] = useState(false);
  const [report, setReport] = useState<OneTouchReport | null>(null);

  const handleComplete = async () => {
    setIsGenerating(true);
    await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy);

    try {
      // En endpoint gör allt
      const result = await api.post(`/api/mobile/orders/${orderId}/complete-with-re-
port`);
      setReport(result.data);

      await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
    } catch (error) {
      Alert.alert('Fel', 'Kunde inte generera rapport');
    } finally {
      setIsGenerating(false);
    }
  };

  return (
    <>
      <TouchableOpacity
        style={styles.completeButton}
        onPress={handleComplete}
        disabled={isGenerating}
      >
        {isGenerating ? (
          <ActivityIndicator color="white" />
        ) : (
          <>
            <Ionicons name="checkmark-circle" size={24} color="white" />
            <Text style={styles.completeButtonText}>KLAR - Skapa Rapport</Text>
          </>
        )}
      </TouchableOpacity>

      <ReportPreviewModal
        visible={!!report}
        report={report}
        onConfirm={() => navigation.goBack()}
        onEdit={() => navigation.navigate('EditReport', { report })}
      />
    </>
  );
}
```

Användarflöden

Flöde 4.1: One-touch completion

```
1. Tekniker har utfört arbete och tagit bilder
2. Trycker stor grön "KLAR"-knapp
3. [AI processar i 5-10 sekunder]:
   - Analyserar alla bilder
   - Sammanställer inspektionsdata
   - Räknar material
   - Beräknar tid
   - Genererar rapport
4. Förhandsgranskning visas:
   ┌──────────────────────────────┐
   │ 📋 RAPPORT GENERERAD         │
   ├──────────────────────────────┤
   │ "Tvätt utförd av 4 matavfallskärl │
   │ och 2 plastkärl. Samtliga i gott  │
   │ skick förutom trasigt lock på     │
   │ kärl #3. Uppgift skapad för byte." │
   │                              │
   │ Tid: 23 min                  │
   │ Material: -                  │
   │ Bilder: 3 st                 │
   │                              │
   │ [✓ Skicka till kund]         │
   │ [✏️ Justera]                 │
   └──────────────────────────────┘
5. Tekniker trycker "Skicka till kund"
6. Rapport sparas, kund notifieras, nästa uppgift visas
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Knapptryckningar	1-2 st	UX-observation
Automatisk ifyllnad	95% av fält	Jämför manuell input
Rapportgenerering	< 10 sekunder	Automatisk mätning
Kvalitet	Granskad av planerare	Feedback-loop

HÖG PRIORITET (Fas 2-3 - Månad 4-9)

Funktion 5: Prediktiv UI

Detaljerad Beskrivning

Systemet lär sig teknikerns beteende och föreslår proaktivt nästa steg:
- “Du tar alltid efterbild här - ta bild?”
- “Brukar du logga material nu?”
- “Nästa uppgift är 5 min bort - navigera?”

Teknisk Implementation

```
// server/services/predictive-ui.ts

interface PredictedAction {
  action: string;
  probability: number;
  context: string;
  trigger: string;
}

export async function getPredictedActions(
  technicianId: string,
  currentState: AppState
): Promise<PredictedAction[]> {
  // Hämta historiskt beteende
  const history = await getTechnicianActionHistory(technicianId, {
    limit: 1000,
    orderStatus: currentState.orderStatus,
    objectType: currentState.objectType,
  });

  // Beräkna sannolikheter för nästa åtgärd
  const predictions = calculateActionProbabilities(history, currentState);

  // Returnera top 3 förslag med hög sannolikhet
  return predictions
    .filter(p => p.probability > 0.7)
    .slice(0, 3)
    .map(p => ({
      action: p.action,
      probability: p.probability,
      context: generateContext(p, history),
      trigger: currentState.orderStatus,
    }));
}

function calculateActionProbabilities(
  history: ActionHistory[],
  state: AppState
): Prediction[] {
  // Filtrera relevanta händelser
  const relevant = history.filter(h =>
    h.orderStatus === state.orderStatus &&
    h.objectType === state.objectType
  );

  // Räkna frekvens för varje åtgärd
  const actionCounts = relevant.reduce((acc, h) => {
    acc[h.nextAction] = (acc[h.nextAction] || 0) + 1;
    return acc;
  }, {} as Record<string, number>);

  const total = relevant.length;

  return Object.entries(actionCounts)
    .map(([action, count]) => ({
      action,
      probability: count / total,
    }))
    .sort((a, b) => b.probability - a.probability);
}
```

React Native Implementation

```
// client/components/PredictiveActions.tsx

export function PredictiveActions({ orderId, orderStatus }) {
  const { data: predictions } = useQuery({
    queryKey: ['predictions', orderId, orderStatus],
    queryFn: () => api.get(`/api/mobile/orders/${orderId}/predictions`),
    refetchInterval: 30000, // Uppdatera var 30:e sekund
  });

  if (!predictions?.length) return null;

  return (
    <View style={styles.predictionsContainer}>
      <Text style={styles.predictionsTitle}>💡 Förslag</Text>
      {predictions.map((p, i) => (
        <TouchableOpacity
          key={i}
          style={styles.predictionCard}
          onPress={() => handlePrediction(p)}
        >
          <View style={styles.predictionContent}>
            <Text style={styles.predictionAction}>{p.action}</Text>
            <Text style={styles.predictionContext}>{p.context}</Text>
          </View>
          <View style={styles.probabilityBadge}>
            <Text>{Math.round(p.probability * 100)}%</Text>
          </View>
        </TouchableOpacity>
      ))}
    </View>
  );
}
```

Användarflöden

Flöde 5.1: Föreslå efterbild

```
1. Tekniker slutför tvätt
2. App detekterar: "95% av gångerna tar du efterbild här"
3. Visar förslag:
   ┌──────────────────────────────┐
   │ 💡 FÖRSLAG                   │
   ├──────────────────────────────┤
   │ 📷 Ta efterbild? (du gör det 95%)  │
   │ 📝 Logga material? (du gör det 30%)│
   │ ➡️ Nästa uppgift? (5 min bort)     │
   └──────────────────────────────┘
4. Tekniker swipear höger på "Ta efterbild"
5. Kamera öppnas direkt
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Prediktion noggrannhet	> 80%	Jämför förslag vs faktisk åtgärd
Förslag accepteras	> 50% av visade	Klickstatistik
Laddtid	< 500ms	Automatisk mätning
Personalisering	Per tekniker	Validera skillnader

Funktion 6: “Daglig Briefing”

Detaljerad Beskrivning

Vid appstart morgonen visar AI en personlig briefing:
- Dagens uppgifter med AI-genererad sammanfattning
- Viktiga koder, nycklar, kontaktinfo
- Väder och trafik
- Förväntad sluttid

Teknisk Implementation

```
// server/services/daily-briefing.ts

export async function generateDailyBriefing(
  technicianId: string,
  date: Date
): Promise<DailyBriefing> {
  const [orders, weather, traffic, specialInfo] = await Promise.all([
    getOrdersForDay(technicianId, date),
    getWeatherForecast(date),
    getTrafficPredictions(date),
    getSpecialInformation(technicianId, date),
  ]);

  // AI genererar sammanfattning
  const summary = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [
      {
        role: 'system',
        content: `
          Generera en kort, vänlig morgonbriefing för en fälttekniker.
          Inkludera:
          1. Antal uppgifter och typ
          2. Speciella utmaningar (väder, trafik)
          3. Viktiga saker att komma ihåg
          Max 4 meningar. Var positiv men realistisk.
        `
      },
      {
        role: 'user',
        content: JSON.stringify({ orders, weather, traffic, specialInfo })
      }
    ]
  });

  return {
    date,
    technicianId,
    summary: summary.choices[0].message.content,
    orders: orders.map(o => ({
      id: o.id,
      address: o.address,
      description: o.description,
      estimatedTime: o.estimatedTime,
      specialCodes: extractSpecialCodes(o),
    })),
    weather: {
      temperature: weather.temperature,
      condition: weather.condition,
      warning: weather.warning,
    },
    traffic: {
      status: traffic.status,
      delayMinutes: traffic.estimatedDelay,
      affectedRoutes: traffic.affectedRoutes,
    },
    estimatedEndTime: calculateEstimatedEndTime(orders, traffic),
    importantNotes: specialInfo,
  };
}
```

React Native Implementation

```
// client/screens/HomeScreen.tsx - Utökad med briefing

export function HomeScreen() {
  const { data: briefing, isLoading } = useQuery({
    queryKey: ['dailyBriefing', today],
    queryFn: () => api.get('/api/mobile/briefing'),
    staleTime: 1000 * 60 * 30, // Cache 30 min
  });

  if (isLoading) return <BriefingLoadingSkeleton />;

  return (
    <ScrollView style={styles.container}>
      {/* AI-genererad briefing */}
      <View style={styles.briefingCard}>
        <View style={styles.briefingHeader}>
          <Text style={styles.greeting}>God morgon, {userName}! ☀️</Text>
          <Text style={styles.date}>{formatDate(today)}</Text>
        </View>

        <Text style={styles.aiSummary}>{briefing.summary}</Text>

        {/* Quick stats */}
        <View style={styles.quickStats}>
          <Stat icon="list" value={briefing.orders.length} label="Uppgifter" />
          <Stat icon="time" value={briefing.estimatedHours} label="Timmar" />
          <Stat icon="car" value={briefing.totalKm} label="Km" />
        </View>

        {/* Väder */}
        <WeatherWidget weather={briefing.weather} />

        {/* Trafik */}
        {briefing.traffic.delayMinutes > 0 && (
          <TrafficWarning traffic={briefing.traffic} />
        )}

        {/* Viktiga koder */}
        {briefing.importantCodes.length > 0 && (
          <ImportantCodesCard codes={briefing.importantCodes} />
        )}
      </View>

      {/* Starta dagen-knapp */}
      <TouchableOpacity style={styles.startButton}>
        <Text style={styles.startButtonText}>STARTA DAGEN</Text>
        <Text style={styles.firstTask}>
          Första: {briefing.orders[0]?.address}
        </Text>
      </TouchableOpacity>

      {/* Uppgiftslista */}
      <OrderList orders={briefing.orders} />
    </ScrollView>
  );
}
```

Användarflöden

Flöde 6.1: Morgonstart

```
1. Tekniker öppnar app kl 07:00
2. Briefing visas:
   ┌──────────────────────────────────────┐
   │ God morgon, Johan! ☀️                 │
   │ Tisdag 25 februari 2026              │
   ├──────────────────────────────────────┤
   │ "Du har 8 uppgifter idag, mestadels  │
   │ tvätt i Sigtuna-området. Vädret ser  │
   │ bra ut men det är halt på vägarna.   │
   │ Kom ihåg ny portkod på Storgatan 5!" │
   │                                      │
   │ ┌──────┐ ┌──────┐ ┌──────┐          │
   │ │ 8    │ │ 6.5h │ │ 45 km │          │
   │ │uppgift│ │ tid  │ │ köra  │          │
   │ └──────┘ └──────┘ └──────┘          │
   │                                      │
   │ 🌡️ 3°C, klart          ⚠️ Halt väglag│
   │                                      │
   │ 🔑 VIKTIGA KODER:                    │
   │ • Storgatan 5: 4523 (NY!)            │
   │ • Palmgatan 17: 5678                 │
   │                                      │
   │ [       STARTA DAGEN       ]         │
   │ Första: Palmgatan 17 (07:30-08:30)   │
   └──────────────────────────────────────┘
3. Tekniker trycker "STARTA DAGEN"
4. Navigation till första uppgift startar
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Briefing-laddtid	< 2 sekunder	Automatisk mätning
AI-sammanfattning	Relevant och korrekt	Användarfeedback
Koder visas	100% av viktiga koder	Validering
Väderdata	Aktuell (< 1h gammal)	API-kontroll

Funktion 7: Smart Kunskapsbas (Mobil-vy)

Detaljerad Beskrivning

Sök med naturligt språk, få AI-genererade svar baserat på:
- Arbetsinstruktioner och manualer
- Objekthistorik
- Erfarna teknikers tips
- FAQ

Teknisk Implementation

```
// server/services/mobile-knowledge.ts

export async function searchKnowledge(
  query: string,
  context: SearchContext
): Promise<KnowledgeResult[]> {
  // 1. Vektor-sökning i Pinecone
  const embedding = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: query,
  });

  const vectorResults = await pinecone.query({
    vector: embedding.data[0].embedding,
    topK: 10,
    filter: { tenantId: context.tenantId },
  });

  // 2. Keyword-sökning i PostgreSQL
  const textResults = await db.query(`
    SELECT * FROM knowledge_base
    WHERE tenant_id = $1
    AND to_tsvector('swedish', content) @@ plainto_tsquery('swedish', $2)
    LIMIT 10
  `, [context.tenantId, query]);

  // 3. Kombinera och re-ranka
  const combined = mergeAndRerank(vectorResults, textResults);

  // 4. AI genererar svar
  const answer = await openai.chat.completions.create({
    model: 'gpt-4o-mini',
    messages: [
      {
        role: 'system',
        content: `
          Du är en hjälpsam assistent för fälttekniker.
          Svara baserat på följande kunskapsdokument:
          ${combined.map(r => r.content).join('\n\n')}

          Var konkret och praktisk. Max 3 meningar.
        `
      },
      { role: 'user', content: query }
    ]
  });

  return {
    answer: answer.choices[0].message.content,
    sources: combined.slice(0, 3).map(r => ({
      title: r.title,
      snippet: r.snippet,
      url: r.url,
    })),
  };
}
```

React Native Implementation

```
// client/screens/KnowledgeSearchScreen.tsx

export function KnowledgeSearchScreen() {
  const [query, setQuery] = useState('');
  const [result, setResult] = useState<KnowledgeResult | null>(null);

  const handleSearch = async () => {
    setIsSearching(true);
    const result = await api.post('/api/mobile/knowledge/search', { query });
    setResult(result.data);
    setIsSearching(false);
  };

  return (
    <View style={styles.container}>
      {/* Sökfält */}
      <View style={styles.searchContainer}>
        <TextInput
          style={styles.searchInput}
          placeholder="Ställ en fråga..."
          value={query}
          onChangeText={setQuery}
          onSubmitEditing={handleSearch}
        />
        <TouchableOpacity style={styles.voiceButton}>
          <Ionicons name="mic" size={24} />
        </TouchableOpacity>
      </View>

      {/* Snabbfrågor */}
      <View style={styles.quickQuestions}>
        <QuickQuestion text="Hur byter jag gasdämpare?" />
        <QuickQuestion text="Var hittar jag reservdelar?" />
        <QuickQuestion text="Vad gör jag vid låst dörr?" />
      </View>

      {/* AI-svar */}
      {result && (
        <View style={styles.answerCard}>
          <Text style={styles.answerTitle}>🤖 Svar</Text>
          <Text style={styles.answerText}>{result.answer}</Text>

          <Text style={styles.sourcesTitle}>Källor:</Text>
          {result.sources.map((s, i) => (
            <TouchableOpacity key={i} style={styles.sourceItem}>
              <Text style={styles.sourceTitle}>{s.title}</Text>
              <Text style={styles.sourceSnippet}>{s.snippet}</Text>
            </TouchableOpacity>
          ))}
        </View>
      )}
    </View>
  );
}
```

Användarflöden

Flöde 7.1: Textfråga

```
1. Tekniker öppnar kunskapssökning
2. Skriver: "Hur byter jag gasdämpare på molok?"
3. [AI söker och genererar svar]
4. Svar visas:
   ┌──────────────────────────────┐
   │ 🤖 SVAR                      │
   ├──────────────────────────────┤
   │ "Öppna locket helt, skruva loss de │
   │ två M8-bultarna på dämparen, lyft  │
   │ bort den gamla och montera den nya │
   │ i omvänd ordning. Kräver 13mm      │
   │ nyckel."                           │
   │                                    │
   │ KÄLLOR:                            │
   │ • Molok Manual v2.3                │
   │ • Tips från Johan (50+ byten)      │
   └──────────────────────────────┘
```

Flöde 7.2: Röstfråga

```
1. Tekniker trycker mikrofon-knapp
2. Säger: "Vad är standardkoden för soprum?"
3. AI svarar (röst + text):
   "Standardkoden för soprum är oftast 1234 eller 0000.
   Men kolla alltid objektets metadata först."
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Svarstid	< 3 sekunder	Automatisk mätning
Relevans	80% användbara svar	Användarfeedback
Källhänvisning	100% av svar har källa	Validering

MEDEL PRIORITET (Fas 4 - Månad 10-12)

Funktion 8: AR-Assisterad Besiktning

Detaljerad Beskrivning

Augmented Reality-guide för besiktning:
- Kameran visar objekt med överlagrade instruktioner
- AI identifierar komponenter och markerar dem
- Checklista visas i AR
- Automatisk dokumentation av inspekterade delar

Teknisk Implementation

```
// client/screens/ARInspectionScreen.tsx

import { ARView } from 'expo-ar';
import * as tf from '@tensorflow/tfjs';

export function ARInspectionScreen({ orderId }) {
  const [detections, setDetections] = useState([]);
  const [checklist, setChecklist] = useState([]);

  // Object detection model
  const model = useTFModel('component-detection-v1');

  const handleFrame = async (frame: CameraFrame) => {
    // Kör objektdetektering
    const predictions = await model.detect(frame);

    // Filtrera relevanta komponenter
    const relevant = predictions.filter(p =>
      ['kärl', 'lock', 'gasdämpare', 'hjul', 'etikett'].includes(p.class)
    );

    setDetections(relevant);
  };

  return (
    <ARView onFrame={handleFrame}>
      {/* Överlagrade markeringar */}
      {detections.map((d, i) => (
        <AROverlay
          key={i}
          position={d.boundingBox}
          label={d.class}
          status={checklist[d.class]?.status || 'unchecked'}
          onTap={() => openInspectionDialog(d)}
        />
      ))}

      {/* Flytande checklista */}
      <ARChecklist items={checklist} onItemTap={handleChecklistItem} />
    </ARView>
  );
}
```

Acceptanskriterier

Kriterie	Mål	Mätmetod
Objektigenkänning	80% i realtid	Test i fält
FPS	> 30 fps	Performance-test
Användarbarhet	Positivt mottagande	Användartester

Funktion 9: Adaptiv UI

Detaljerad Beskrivning

Gränssnitt anpassar sig automatiskt baserat på:
- Teknikerns vanligaste arbetsflöden
- Tid på dagen (morgon vs eftermiddag)
- Typ av uppgift
- Tidigare beteende

Teknisk Implementation

```
// server/services/adaptive-ui.ts

export async function getAdaptiveUIConfig(
  technicianId: string,
  context: UIContext
): Promise<UIConfig> {
  const behaviorProfile = await getTechnicianBehaviorProfile(technicianId);

  return {
    // Anpassa ordning på knappar
    buttonOrder: behaviorProfile.mostUsedActions.slice(0, 4),

    // Anpassa tema
    theme: context.timeOfDay === 'evening' ? 'dark' : 'light',

    // Anpassa komplexitet
    complexity: behaviorProfile.techLevel, // 'simple' | 'standard' | 'advanced'

    // Visa/dölj sektioner
    sections: {
      weather: behaviorProfile.checksWeather > 0.5,
      traffic: behaviorProfile.checksTraffic > 0.3,
      tips: behaviorProfile.experienceLevel < 6, // månader
    },

    // Snabbkommandon
    shortcuts: behaviorProfile.frequentPaths.slice(0, 3),
  };
}
```

Funktion 10: Smart Notifieringar

Detaljerad Beskrivning

Kontextbaserade notiser som:
- Prioriterar baserat på brådska
- Grupperar relaterade meddelanden
- Anpassar sig till aktivitetsnivå
- Tystar sig under körning

Teknisk Implementation

```
// server/services/smart-notifications.ts

export async function sendSmartNotification(
  technicianId: string,
  notification: Notification
): Promise<NotificationResult> {
  const context = await getTechnicianContext(technicianId);

  // Bestäm prioritet
  const priority = calculatePriority(notification, context);

  // Kontrollera om vi ska tysta
  if (context.isDriving && priority < Priority.URGENT) {
    return await queueNotification(notification, context.estimatedArrival);
  }

  // Gruppera med liknande
  const grouped = await groupWithSimilar(notification, technicianId);

  // Anpassa format
  const formatted = formatForContext(grouped, context);

  return await sendPushNotification(technicianId, formatted);
}
```

---

4. Teknisk Arkitektur

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          FÄLT APP ARKITEKTUR                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                    MOBIL APP (React Native/Expo)                 │   │
│   ├──────────────────────────────────────────────────────────────────┤   │
│   │  Screens          │  Components           │  Services            │   │
│   │  • HomeScreen     │  • VoiceAssistant     │  • voice-assistant.ts │   │
│   │  • OrdersScreen   │  • SmartCamera        │  • image-analysis.ts  │   │
│   │  • ARInspection   │  • PredictiveActions  │  • offline-sync.ts    │   │
│   │  • Knowledge      │  • DailyBriefing      │  • location.ts        │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                   │                                      │
│                                   ▼                                      │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                        API GATEWAY                               │   │
│   │                    (Express.js + TypeScript)                     │   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                   │                                      │
│           ┌───────────────────────┼───────────────────────┐              │
│           ▼                       ▼                       ▼              │
│   ┌──────────────┐          ┌──────────────┐          ┌──────────────┐  │
│   │ AI SERVICES  │          │ BUSINESS     │          │ EXTERNAL     │  │
│   ├──────────────┤          │ SERVICES     │          │ APIs         │  │
│   │ • Voice      │          ├──────────────┤          ├──────────────┤  │
│   │   (Whisper)  │          │ • Orders     │          │ • Google Maps │  │
│   │ • Vision     │          │ • Metadata   │          │ • Open-Meteo  │  │
│   │   (GPT-4V)   │          │ • Reports    │          │ • Twilio      │  │
│   │ • NLP        │          │ • GPS        │          │ • ElevenLabs  │  │
│   │   (GPT-4)    │          │ • Knowledge  │          │ • Pinecone    │  │
│   │ • TTS        │          │              │          │               │  │
│   │   (Eleven)   │          │              │          │               │  │
│   └──────────────┘          └──────────────┘          └──────────────┘  │
│                                   │                                      │
│                                   ▼                                      │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │                          DATA LAYER                              │   │
│   │  PostgreSQL (main) │ Redis (cache) │ Pinecone (vectors) │ S3 (files)│   │
│   └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

5. API-Integrationer

OpenAI API

Modell	Användning	Kostnad/1K tokens
Whisper	Speech-to-Text	$0.006/min
GPT-4o	Reasoning, Vision	$0.005 input, $0.015 output
GPT-4o-mini	Snabba frågor	$0.00015 input, $0.0006 output
text-embedding-3-small	Vektorsökning	$0.00002

ElevenLabs

Funktion	Plan	Kostnad
Text-to-Speech	Creator	$22/mån (30K tecken)
Svenska röster	Ingår	-

Google Maps

API	Användning	Kostnad
Directions	Rutt + ETA	$0.005/request
Distance Matrix	Restidsberäkning	$0.005/element

Twilio

Tjänst	Användning	Kostnad
SMS (Sverige)	Kundnotiser	$0.075/SMS

Pinecone

Plan	Användning	Kostnad
Starter	Kunskapsbas	$0.00/mån (begränsad)
Standard	Produktion	$70/mån

---

6. Datamodell (Nya tabeller)

```sql
-- Voice Assistant History
CREATE TABLE voice_interactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    technician_id UUID REFERENCES users(id),
    order_id UUID REFERENCES work_orders(id),
    transcript TEXT NOT NULL,
    response TEXT NOT NULL,
    actions_taken JSONB,
    latency_ms INTEGER,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Image Analysis Results
CREATE TABLE image_analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    image_id UUID REFERENCES images(id),
    order_id UUID REFERENCES work_orders(id),
    analysis_result JSONB NOT NULL,
    objects_detected JSONB,
    anomalies_detected JSONB,
    confidence FLOAT,
    metadata_updates JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Predictive UI Data
CREATE TABLE technician_behavior (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    technician_id UUID REFERENCES users(id),
    action_type VARCHAR(50),
    context JSONB,
    timestamp TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_technician_behavior ON technician_behavior(technician_id, ac-
tion_type);

-- Knowledge Base
CREATE TABLE knowledge_base (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id),
    title VARCHAR(255),
    content TEXT,
    content_vector VECTOR(1536),
    source VARCHAR(100),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_knowledge_content ON knowledge_base USING GIN(to_tsvector('swedish',
content));
```

7. Användarflöden (Detaljerade)

Flöde 1: Morgonstart med AI-Briefing

```
╔═══════════════════════════════════════════════════════════════════════╗
║                    MORGONSTART - KOMPLETT FLÖDE                       ║
╚═══════════════════════════════════════════════════════════════════════╝
│                                                                       │
│  07:00 - Tekniker öppnar app                                          │
│    │                                                                  │
│    ▼                                                                  │
│  ┌─────────────────────────────────────────────────────────────────┐  │
│  │ App laddas → API /briefing anropas → AI genererar sammanfattning │  │
│  └─────────────────────────────────────────────────────────────────┘  │
│    │                                                                  │
│    ▼                                                                  │
│  07:01 - Briefing visas                                               │
│    │   • Dagens 8 uppgifter                                           │
│    │   • Väder: 3°C, halt                                             │
│    │   • Trafikläge: Normalt                                          │
│    │   • Viktiga koder markerade                                      │
│    │                                                                  │
│    ▼                                                                  │
│  07:02 - Tekniker trycker "STARTA DAGEN"                              │
│    │                                                                  │
│    ▼                                                                  │
│  ┌─────────────────────────────────────────────────────────────────┐  │
│  │ • GPS-spårning aktiveras                                         │  │
│  │ • Status uppdateras till "on_duty"                               │  │
│  │ • Första uppgift markeras som "assigned"                         │  │
│  │ • Navigation till första adress startar (Google Maps)            │  │
│  │ • Kund får SMS: "Johan är på väg, ankomst 07:30"                 │  │
│  └─────────────────────────────────────────────────────────────────┘  │
│                                                                       │
╒═══════════════════════════════════════════════════════════════════════╕
```

Flöde 2: Utföra Uppgift med Röstassistent

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                      UPPGIFT MED RÖSTASSISTENT                            ╒
╟═══════════════════════════════════════════════════════════════════════════╢
│                                                                           │
│  07:30 - Tekniker anländer (GPS detekterar)                               │
│    │                                                                      │
│    ├──► Status automatiskt → "arrived"                                    │
│    ├──► Kund får SMS: "Johan har anlänt!"                                 │
│    │                                                                      │
│    ▼                                                                      │
│  07:31 - Tekniker: 🎤 "Vad är koden till dörren?"                         │
│    │                                                                      │
│    ├──► [Whisper transkriberar]                                           │
│    ├──► [GPT-4 hämtar metadata: portkod = "5678"]                         │
│    ├──► [ElevenLabs genererar svar]                                       │
│    │                                                                      │
│    ▼                                                                      │
│  07:31 - AI: 🔊 "Portkoden är 5678. Den uppdaterades i november."         │
│    │                                                                      │
│    ▼                                                                      │
│  07:35 - Tekniker utför arbete, ser trasigt lysrör                        │
│    │                                                                      │
│    ├──► 🎤 "Jag ser ett trasigt lysrör"                                   │
│    │                                                                      │
│    ▼                                                                      │
│  AI: 🔊 "Jag noterar det. Vill du ta bild och skapa uppgift för byte?"    │
│    │                                                                      │
│    ├──► 🎤 "Ja"                                                           │
│    │                                                                      │
│    ▼                                                                      │
│  [Kamera öppnas automatiskt] → Tekniker tar bild → [Vision AI analyserar] │
│    │                                                                      │
│    ▼                                                                      │
│  AI: 🔊 "Bild sparad. Jag ser LED-lysrör 60cm. Uppgift 'Byt lysrör'       │
│         skapad på Palmgatan 17. Vill du lägga till på din rutt nästa vecka?" │
│    │                                                                      │
│    ├──► 🎤 "Ja, gärna"                                                    │
│    │                                                                      │
│    ▼                                                                      │
│  AI: 🔊 "Klart! Inbokat torsdag nästa vecka."                             │
│                                                                           │
╚═══════════════════════════════════════════════════════════════════════════╝
```

Flöde 3: Automatisk Rapportering med Bildanalys

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        AUTOMATISK RAPPORTERING                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  07:50 - Arbete klart, tekniker tar 3 bilder                             │
│    │                                                                     │
│    ├──► Bild 1: Före (togs vid start)                                    │
│    ├──► Bild 2: Under arbete                                             │
│    ├──► Bild 3: Efter (tas nu)                                           │
│    │                                                                     │
│    ▼                                                                     │
│  [Vision AI analyserar automatiskt]                                      │
│    │                                                                     │
│    ├──► Objektigenkänning: "4 kärl identifierade (inte 1 som angivet)"   │
│    ├──► Avvikelse: "Trasigt lock på kärl #3"                             │
│    ├──► Före/efter: "Synlig förbättring, kärl rengjorda"                 │
│    │                                                                     │
│    ▼                                                                     │
│  07:51 - App visar analys:                                               │
│    │                                                                     │
│    │   ┌──────────────────────────────────────────────────────────┐     │
│    │   │ 🤖 AI-ANALYS KLAR                                        │     │
│    │   ├──────────────────────────────────────────────────────────┤     │
│    │   │ ✅ Identifierat: 4 kärl (matavfall 240L x2, plast x2)    │     │
│    │   │ ⚠️ Avvikelse: Metadata säger 1 kärl → uppdatera till 4?  │     │
│    │   │ ⚠️ Upptäckt: Trasigt lock på kärl #3                     │     │
│    │   │ ✅ Före/efter: Tydlig förbättring                         │     │
│    │   │                                                          │     │
│    │   │ [✓ Godkänn allt] [✏️ Justera] [Ta ny bild]               │     │
│    │   └──────────────────────────────────────────────────────────┘     │
│    │                                                                     │
│    ▼                                                                     │
│  07:52 - Tekniker trycker "Godkänn allt"                                 │
│    │                                                                     │
│    ├──► Metadata uppdateras: antal_karl = 4                              │
│    ├──► Rapport genereras automatiskt                                    │
│    ├──► Uppgift "Byt lock" skapas                                        │
│    ├──► Kund får SMS med rapport: "Tvätt klar! 4 kärl tvättade..."       │
│    │                                                                     │
│    ▼                                                                     │
│  07:52 - Nästa uppgift visas automatiskt                                 │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Flöde 4: Prediktiv UI - Nästa Steg

```
╔═══════════════════════════════════════════════════════════════════════════╗
║                          PREDIKTIV UI                                     ║
╟═══════════════════════════════════════════════════════════════════════════╢
│                                                                           │
│  Efter varje statusändring, analyserar AI teknikerns vanliga mönster:     │
│                                                                           │
│  [Status: "completed"]                                                    │
│    │                                                                      │
│    ├──► AI analyserar: "Johan tar efterbild i 95% av fallen"              │
│    ├──► AI analyserar: "Johan loggar sällan material (15%)"               │
│    ├──► AI analyserar: "Johan navigerar ofta till nästa direkt"           │
│    │                                                                      │
│    ▼                                                                      │
│  App visar prediktiva förslag:                                            │
│    │                                                                      │
│    │   ┌─────────────────────────────────────────────────────────┐       │
│    │   │ │ 💡 FÖRSLAG                                            │ │     │
│    │   ├─────────────────────────────────────────────────────────┤       │
│    │   │ │ [📷 Ta efterbild?      ] (du gör det 95% av gångerna) ●●●●● │ │     │
│    │   │ │ [➡️ Nästa uppgift?     ] (5 min, Storgatan 5)         ●●●●○ │ │     │
│    │   │ │ [📝 Logga material?    ] (du gör det 15%)             ●○○○○ │ │     │
│    │   └─────────────────────────────────────────────────────────┘       │
│    │                                                                      │
│    ▼                                                                      │
│  Tekniker sveper höger på "Ta efterbild" → Kamera öppnas direkt           │
│                                                                           │
╚═══════════════════════════════════════════════════════════════════════════╝
```

---

8. UI/UX-Design

Designprinciper

Princip	Implementation
Minimalism	Max 3 huvudåtgärder per skärm
Stora träffytor	Knappar minst 48x48px, för handskar
Hög kontrast	Fungerar i direkt solljus
Mörkt läge	Automatiskt baserat på tid/ljus
Offline-först	Alla kritiska funktioner fungerar offline
Hands-free	Röststyrning som primär input

Fältanpassad Design

```
┌──────────────────────────────────────────────────────────────────────┐
│                        FÄLTANPASSAD UI-DESIGN                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────────────────────────┐                       │
│   │ STATUS BAR (alltid synlig)               │                       │
│   │ • Offline-indikator (röd/grön)           │                       │
│   │ • Batterinivå                            │                       │
│   │ • GPS-status                             │                       │
│   │ • Tid                                    │                       │
│   └──────────────────────────────────────────┘                       │
│                                                                      │
│   ┌──────────────────────────────────────────┐                       │
│   │ CONTENT AREA (scrollbar)                 │                       │
│   │                                          │                       │
│   │ • Stora kort med hög kontrast            │                       │
│   │ • Minimalt med text                      │                       │
│   │ • Ikoner + färgkodning                   │                       │
│   │ • Swipe-gester för snabba åtgärder       │                       │
│   │                                          │                       │
│   └──────────────────────────────────────────┘                       │
│                                                                      │
│   ┌──────────────────────────────────────────┐                       │
│   │ BOTTOM ACTION BAR (fast)                 │                       │
│   │                                          │                       │
│   │  ┌────┐   ┌────┐   ┌────┐   ┌────┐      │                       │
│   │  │ 🏠 │   │ 📋 │   │ 🎤 │   │ 👤 │      │                       │
│   │  │Home│   │Order│  │Voice│  │Prof │      │                       │
│   │  └────┘   └────┘   └────┘   └────┘      │                       │
│   └──────────────────────────────────────────┘                       │
│                                                                      │
│   ┌──────────────────────────────────────────┐                       │
│   │ FLOATING ACTION BUTTON (röstassistent)   │                       │
│   │                                          │                       │
│   │                          ┌────┐          │                       │
│   │                          │ 🎤 │          │                       │
│   │                          └────┘          │                       │
│   │                      (håll för röst)     │                       │
│   └──────────────────────────────────────────┘                       │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Färgschema

```
/* Fält App Theme */
:root {
  /* Primary */
  --primary: #1B4F72;
  --primary-light: #2874A6;
  --primary-dark: #154360;

  /* Secondary */
  --secondary: #17A589;
  --secondary-light: #1ABC9C;
  --secondary-dark: #148F77;

  /* Status */
  --success: #27AE60;
  --warning: #F39C12;
  --error: #E74C3C;
  --info: #3498DB;

  /* Neutral */
  --background: #FFFFFF;
  --surface: #F8F9FA;
  --text-primary: #2C3E50;
  --text-secondary: #7F8C8D;

  /* Dark mode */
  --dark-background: #1A1A2E;
  --dark-surface: #16213E;
  --dark-text-primary: #EAECEE;
}
```

---

9. Implementationsordning (Fas 1)

Vecka 1-2: Setup och Grundfunktionalitet

- [ ] **Dag 1-2: Projekt-setup**
- Skapa ny branch `feature/ai-integration`
- Installera nya beroenden (OpenAI SDK, ElevenLabs SDK)
- Konfigurera miljövariabler
- Skapa nya API-routes skelett
- [ ] **Dag 3-5: Backend AI-services**
- Implementera `voice-assistant.ts` service
- Implementera `vision-analysis.ts` service
- Implementera `customer-communication.ts` service
- Skapa API-endpoints
- [ ] **Dag 6-10: Database och migrations**
- Skapa nya tabeller (voice_interactions, image_analyses, etc.)
- Indexering och optimering
- Seed data för test

Vecka 3-4: Röstassistent (MVP)

- [ ] **Dag 11-14: React Native Voice**
- Implementera `VoiceAssistant.tsx` komponent
- Ljud-inspelning med `expo-av`
- Push-to-talk UI
- Haptisk feedback
- [ ] **Dag 15-18: Backend Voice Processing**
- Whisper API integration
- GPT-4 kontexthantering
- ElevenLabs TTS
- Streaming response
- [ ] **Dag 19-20: Test och validering**
- Test i verklig miljö
- Bullertolerans-test
- Latenstester
- Användartest (2-3 tekniker)

Vecka 5-6: Bildanalys (MVP)

- [ ] **Dag 21-24: Kamera och Upload**
- Förbättra `CameraCaptureScreen.tsx`
- Bildkomprimering
- Upload-pipeline
- [ ] **Dag 25-28: Vision AI**
- GPT-4V integration
- Objektigenkänning
- Avvikelsedetektering
- Metadata-uppdatering
- [ ] **Dag 29-30: Test och validering**
- Noggrannhetstest (100 bilder)
- Hastighetstester
- Edge cases

Vecka 7-8: One-Touch Reporting

- [ ] **Dag 31-34: Rapport-generering**
- Implementera `one-touch-report.ts`
- AI-sammanfattning
- PDF-generering
- [ ] **Dag 35-38: UI Integration**
- “KLAR”-knapp
- Förhandsgransknings-modal

- Kundkommunikation
- [ ] **Dag 39-40: Test**
- End-to-end tester
- Användartest

Vecka 9-10: Daglig Briefing

- [ ] **Dag 41-44: Briefing-service**
- Implementera `daily-briefing.ts`
- AI-sammanfattning
- Väder och trafik-integration
- [ ] **Dag 45-48: HomeScreen Update**
- Briefing-vy
- “Starta dagen”-flöde
- Animationer
- [ ] **Dag 49-50: Test**
- Morgon-tester med riktiga data
- Performance-optimering

Vecka 11-12: Integration och Test

- [ ] **Dag 51-54: Integration**
- Integration med Kinab Core
- Synkronisering av data
- Event-driven kommunikation
- [ ] **Dag 55-58: Komplett testning**
- End-to-end tester
- Lasttest
- Säkerhetstest
- GDPR-compliance
- [ ] **Dag 59-60: Beta-release**
- Deploy till testmiljö
- 5 tekniker i beta
- Feedback-insamling
- Prioritering av bugfixar

---

10. Acceptanskriterier (Fas 1 - Komplett)

Röstassistent

#	Kriterie	Mål	Test
1	Rösttranskription	95% korrekt	100 testfraser
2	Total svarstid	< 3 sekunder	Automatisk mätning
3	Bullerhantering	Fungerar vid 70+ dB	Fälttest
4	Hands-free operation	Ingen touch efter aktivering	Observationstest
5	Kontextförståelse	90% korrekta svar	Manuell validering
6	Åtgärdsutförande	100% korrekta åtgärder	Automatisk validering
7	Svenskt tal	Naturligt uttal	Användarfeedback (>4/5)
8	Wake word	“Hej Unicorn” fungerar	50 testaktivieringar
9	Offline fallback	Meddelande visas	Test utan nätverk
10	Batteripåverkan	< 5% extra per timme	Batterimätning

Bildanalys

#	Kriterie	Mål	Test
1	Objektigenkänning	85% noggrannhet	100 bilder, jämför manuell räkning
2	Feldetektering	80% upptäckt	50 bilder med kända fel
3	Analysttid	< 5 sekunder	Automatisk mätning
4	Metadata-uppdater-ing	100% korrekt format	Schema-validering
5	Före/efter-jämförelse	90% korrekta slutsat-ser	30 bildpar
6	Text-igenkänning (OCR)	80% korrekt	20 bilder med text
7	Bildkomprimering	< 500KB	Storlekstest
8	Offline köning	Bilder sparas	Test utan nätverk

One-Touch Reporting

#	Kriterie	Mål	Test
1	Knapptryckningar	1-2 st	UX-observation
2	Automatisk ifyllnad	95% av fält	Jämför manuell input
3	Rapportgenerering	< 10 sekunder	Automatisk mätning
4	Kvalitet	Godkänd av planer-are	Feedback 20 rapport-er
5	Kundnotis	Skickas inom 30 sek	Automatisk mätning
6	PDF-kvalitet	Professionell	Visuell granskning

Daglig Briefing

#	Kriterie	Mål	Test
1	Laddtid	< 2 sekunder	Automatisk mätning
2	AI-sammanfattning	Relevant och korrekt	Användarfeedback
3	Koder visas	100% av viktiga koder	Validering mot metadata
4	Väderdata	Aktuell (< 1h gammal)	API-tidstämpel
5	ETA-noggrannhet	±10%	Jämför med faktisk

---

11. Tekniska Krav

Plattform

- **iOS:** 13.0+ (iPhone 8 och nyare)
- **Android:** API 26+ (Android 8.0)
- **Expo SDK:** 54+
- **React Native:** 0.74+

Prestanda

- App-start: < 2 sekunder
- Skärmövergång: < 300ms
- API-svar: < 500ms (exkl. AI)
- Minnesanvändning: < 200MB
- CPU vid vila: < 2%

Offline

- Alla uppgifter tillgängliga offline
- Bilder köas för uppladdning
- Röstkommandon begränsade offline
- Synkronisering vid återanslutning

Säkerhet

- JWT-autentisering
- End-to-end kryptering för röstdata
- Biometrisk upplåsning (valfritt)
- Automatisk utloggning (30 min inaktivitet)

---

12. Säkerhet och Integritet

GDPR-Compliance

Data	Hantering	Retention
Röstinspelningar	Transkriberas och raderas	24h max
Bilder	Lagras på kundens server	Per kundavtal
GPS-positioner	Lagras för spårning	90 dagar
Användarbeteende	Anonymiseras	1 år

Datakryptering

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              KRYPTERING                                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Mobil App] ──────────────────────────────────────────────► [Backend]  │
│       │                  TLS 1.3 (HTTPS)                        │        │
│       │                                                         │        │
│       ▼                                                         ▼        │
│  ┌──────────────┐                                   ┌──────────────┐    │
│  │ Lokal        │                                   │ Server       │    │
│  │ kryptering   │                                   │ kryptering   │    │
│  │ AES-256      │                                   │ at-rest      │    │
│  └──────────────┘                                   └──────────────┘    │
│                                                                          │
│  Röstdata: Krypteras under överföring, raderas efter transkription       │
│  Bilder: Krypteras under överföring, lagras krypterat                    │
│  Metadata: Krypteras under överföring                                    │
└──────────────────────────────────────────────────────────────────────────┘
```

13. Performance-Krav

Funktion	Mål	Max
App-start (cold)	2s	4s
App-start (warm)	500ms	1s
Röstassistent (total)	3s	5s
Bildanalys	5s	10s
Rapportgenerering	5s	15s
Briefing-laddning	2s	4s
API-svar (enkel)	200ms	500ms
Offline sync	30s	60s

---

14. Test och Validering

Teststrategi

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      TESTPYRAMID                                         │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│                          ┌─────────┐                                    │
│                          │  E2E    │  10% - Kritiska flöden             │
│                          │  Test   │                                    │
│                         ┌─────────┐                                     │
│                         │Integration │  20% - API och service-integration │
│                         │   Test  │                                     │
│                        ┌─────────┐                                      │
│                        │   Unit  │  70% - Komponenter och funktioner    │
│                        │   Tests │                                      │
│                        └─────────┘                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Testverktyg

Typ	Verktyg
Unit tests	Jest + React Native Testing Library
Integration	Jest + MSW (Mock Service Worker)
E2E	Detox
Performance	React Native Performance Monitor
Accessibility	axe-react-native

Testmål

- Unit test coverage: **80%**
- Integration test coverage: **60%**
- E2E kritiska flöden: **100%**
- Accessibility score: **AA**

---

15. Dokumentation

Utvecklardokumentation

- [ ] API-dokumentation (Swagger/OpenAPI)
- [ ] Komponentbibliotek (Storybook)
- [ ] Arkitekturbeskrivning
- [ ] Setup-guide

Användardokumentation

- [ ] Användarmanual (PDF)
- [ ] Video-tutorials (5 st)
- [ ] In-app hjälp
- [ ] FAQ

Teknisk dokumentation

- [ ] Dataflödesdiagram
- [ ] Sekvensbeskrivningar
- [ ] AI-promptbibliotek
- [ ] Troubleshooting-guide

---

16. Framtida Utveckling (Fas 2+)

Fas 2 (Månad 4-6)

- Prediktiv UI

- Smart kunskapsbas
- Förbättrad offline

Fas 3 (Månad 7-9)

- AR-besiktning (prototyp)
- Adaptiv UI
- Avancerad analys

Fas 4 (Månad 10-12)

- AR-besiktning (produktion)
- Smart notifieringar
- Full AI-autonomi

Sammanfattning för Replit-utvecklare

Huvudfokus Fas 1:
1. **Röstassistent** - Whisper + GPT-4 + ElevenLabs
2. **Bildanalys** - GPT-4V med automatisk metadata
3. **One-Touch Reporting** - AI-genererade rapporter
4. **Daglig Briefing** - AI-sammanfattning vid start

Tech stack:
- React Native + Expo SDK 54
- Express.js backend
- OpenAI API (Whisper, GPT-4, Vision)
- ElevenLabs TTS
- PostgreSQL + Redis + Pinecone

Kritiska framgångsfaktorer:
- Svarstid < 3 sekunder
- Fungerar med handskar
- Fungerar i buller (70+ dB)
- 80%+ offline-funktionalitet

Dokumentversion: 1.0
Skapad: 2026-02-21
Författare: DeepAgent